CS6160 Complexity

Topic: Syllabus. Turing Machines. Computability. Date: Aug 26, 2026
Lecturer: Wei-Kai Lin Scriber: Wei-Kai Lin

---

1 Syllabus

We go through the syllabus; see https://weikailin.github.io/cs6160-complexity/syllabus/. It covers the goals of this course, the prerequisites, and communication and coursework. Although this course is formally called Theory of Computation at UVA, it is a graduate-level course. Its materials and topics cover a lot in the field of computational complexity at any university, and thus we more often refer to this course as "Complexity."

Here is a brief recap of the course goals. Selfishly, Wei-Kai's goal is to learn the topics systematically through teaching. More generally, the goals of complexity are often to:

- Formalize computation models and resources, so that we can discuss and think about computation more easily while reasoning about or predicting real-world problems with good accuracy.
- Understand the feasibility and limitations of the models and resources.
- Find potential ways to circumvent the limitations, e.g., using randomness or approximation.
- Reason about proving and verifying computation.
- Understand the benefits of having hard problems, such as cryptography.

The coursework consists of presenting course materials, scribing lecture notes, and two quizzes. The presentation and scribed notes are graded. Additionally, the presenter and the scriber are expected to actively participate in the class discussion, such as asking and answering questions, and this participation is graded. The quizzes are in person to assess progress and last 20 minutes each.

Traditionally, in this course, people learn by solving practice problems with pen and paper, just like in math. Because Wei-Kai has limited time and he doesn't like AI-generated solutions, there is no graded homework. We will release optional practice problems: students may turn in the solutions, and Wei-Kai will give feedback but no grades.

2 Turing Machines

The technical materials begin by defining *the* computation model, that is, Turing Machines. Still, as suggested by Arora-Barak in the chapter title [AB09, Chapter 1], "it doesn't matter" which computation model we use. Introducing Turing Machines (TM) in this course mainly serves two purposes: (1) to ensure we have a well-defined model to be precise and clear when we want to, and (2) to familiarize everyone with the same terminologies and notations, such as sets, functions, tuples, sequences, and representations of objects as binary strings, and of course computation.

We often abuse the terminology of Turing Machine, which may refer to the model or a concrete program in the model. It depends on the context, but we often mean the program when we say

"Turing Machine," "TM," or "machine" unless we explicitly say "model."

Historically, in the 1930s, Turing proposed the Turing Machine model [Tur37], slightly after Church's λ-calculus model [Chu36]. The two models are proven to be equivalent, that is, any TM computation can also be done by λ-calculus, and vice versa. Moreover, the equivalence is efficiently constructible: we have an efficient algorithm that, given any TM, outputs an equivalent λ-calculus program, and vice versa.

We typically define $\mathbb{N} = \{0, 1, 2, \dots\}$ and $[n] = \{1, 2, \dots, n\}$ for $n \in \mathbb{N}$ greater than 1. Let $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ be the set of integers.

2.1 Definition

A Turing Machine is a conceptual machine that consists of $k \geq 2$ *tapes*, a finite set Γ of symbols, a finite set $Q = [n]$ of states for some $n \in \mathbb{N}$, and a transition function $\delta : Q \times \Gamma^k \to Q \times \Gamma^{k-1} \times \{L, R, S\}$. Each tape is one-dimensional, discrete, and infinite; that is, a tape consists of locations $i \in \mathbb{Z}$, and each location stores exactly one symbol in Γ. Hence, the tuple (k, Γ, Q, δ) fully specifies a TM.

A TM keeps one *head* for each of k tapes, and the head reads or writes symbols on the tape. To execute a TM, we repeatedly apply the transition function δ on the current state $q \in Q$ and the current symbols $(\gamma_1, \dots, \gamma_k)$ on the k heads, which gives the next state q', the symbols to write on the $k-1$ tapes $(\gamma'_2, \dots, \gamma'_k)$, and then the 'L'eft, 'R'ight, or 'S'tay directions to move the k tapes. The left or right move is exactly one unit of distance along the tape. The first tape is defined to be the *input tape* that stores the input to the TM, and it is read-only, so there is no write symbol for tape 1 in δ, but there are write symbols $(\gamma'_2, \dots, \gamma'_k)$ for tapes from 2 to k. All other tapes are initialized by the blank symbol $\square$, and the input tape stores the n-bit input string. Whenever the transition δ arrives at the halting state $q_{\text{halt}} = 0 \in Q$, we stop the execution and let the non-blank content in the output tape be the output, where the output tape is defined to be the kth tape. See [AB09, Section 1.2] for more details.

A typical choice of symbols is $\Gamma = \{0, 1, \square, \triangleright\}$, and a typical choice of k is 3, where $\triangleright$ is a symbol put on the location 0 for all tapes. They are just easier to write TM programs, as elaborated next, their capabilities are identical.

Importance of Turing Machine model. TM is called the computation model for two reasons. First, Turing Machine is as strong as all other computation models we human have proposed. That includes all modern programming languages, hypothetical computation processes, and even quantum computing. Second, TM is arguably the simplest model that achieves the first point. This makes it easy to show that another computation model can simulate any TM. For example, it is easy to implement a Turing Machine interpreter using modern programming languages.

A fun example is that we can simulate TMs using Magic the Gathering cards [CBH20] and Minesweeper [Kay07]. Of course, we need infinitely many cards or blocks to really simulate it.

2.2 Simulation: More Symbols and Tapes

Suppose that we have a TM M that uses a large alphabet, say Γ consists of letters a-z, digits 0-9, and $\square, \triangleright$. We can build another TM M' that performs the same computation as M. To do so, all letters and digits in the input and output and the internal computation of M are encoded using binary strings. Then, we just program $M' = (k, \Gamma', Q', \delta')$ so that it reads/writes individual bits

but remembers the letter or digit in its state Q'. The running time of M' is $O(\log |\Gamma|)$ times longer then M [AB09, Claim 1.5].

Building the identical functionality using another algorithm, a Turing Machine, or even another computation model is called *simulation*.

Similarly, given any $k \geq 2$ tapes Turing Machine M, we can simulate M by a one-tape Turing Machine. See [AB09, Claim 1.6].

2.3 Two-symbol Machines and Busy Beavers

Deebak asked the following question: Do we really need four symbols in the alphabet Γ?

The answer is that we need at least two symbols to encode meaningful information; indeed, two symbols are sufficient. To see why, observe that symbols are just names, so we can choose any two. Let $\Gamma = \{1, ■\}$ be the two. We encode **0** by ■1, **1** by 1■, □ by ■■, and then ▷ by 11. We let ■ be the default black symbol in the two-symbol machines, so the encoding coincides with the default blank □ in the four-symbol machines. Once we get four symbols, we just need to modify the states and transitions properly.

The above may sound crazy, but there is a famous sequence of Turing Machines defined over two symbols. They are *Busy Beavers*. Formally, for each $n \in \mathbb{N}$, the nth Busy Beaver is the Turing Machine such that, on input 0, it runs longer than all other machines but halts among the set of all two-symbol and n-state and one-tape Turing Machines. Recall that for any fixed n, the number of such n-state machines is finite: there are $2n$ different inputs to δ, each may choose among $6n$ outputs. Also, recall that many TMs never halt on input 0. Hence, finding the Busy Beaver for $n = 5$ was hard, just solved in 2024: https://www.quantamagazine.org/amateur-mathematicians-find-fifth-busy-beaver-turing-machine-20240702/. Notice that the definition of Busy Beaver uses a different halting condition: instead of a state $q_{\text{halt}} \in Q$, the transition function includes H for halting in addition to $\{L, R, S\}$, which just offsets n by 1.

2.4 Universal TM and Oblivious TM

We did not get time to discuss universal Turing Machines. The point is that every TM can be represented by a binary string in a canonical way, e.g., writing the machine in a small set of at most 2^8 characters, 0-9, {}, (), ..., and then encode each character using 8 bits. With that, each binary string $\alpha \in \{0,1\}^*$ represent at most one Turing Machine, denoted by M_α, and if α is not mapped by any TM using the encoding, we define M_α to be a fixed TM, say the 1-state TM that halts immediately.

With that, it is doable to construct a *universal Turing Machine* U such that for all strings $\alpha \in \{0,1\}^*$, we have U simulates M_α, i.e., for all $x \in \{0,1\}^*$, $U(\alpha, x) = M_\alpha(x)$. See [AB09, Section 1.4] for details. See also [AB09, Section 1.7] for a cool and efficient construction of universal TM.

Nathaneal asked about *oblivious Turing Machines*. In general, given any TM M, a TM M' is oblivious if M' simulates M and additionally the heads' moves during the computation $M'(x)$ is identical for all x of the same length. That is, the heads' moves entirely determined by the length $|x|$ but not other information about x. That implies that M should have a time bound $T(n)$ for all input length n; otherwise, identical moves is not well-defined. There is [AB09, Remark 1.7] on oblivious TM. The running time of M' can be optimized to $O(T(n) \cdot \log T(n))$. Wei-Kai does not

remember how to simulate it but suspects that is similar to the universal TM described in [AB09, Section 1.7] because they have the same $T \log T$ asymptotics.

3 Computability

Computability is defined for functions or problems. See also [AB09, Definition 1.3].

Definition 1 (Computable functions)**.** Let $f : \{0,1\}^* \to \{0,1\}$ be a boolean function. We say f is *computable* iff there exists a Turing Machine M such that for all $x \in \{0,1\}^*$, $M(x) = f(x)$. Moreover, for any $T : \mathbb{N} \to \mathbb{N}$, we say that M computes f in time $T(n)$ if for any $n \in \mathbb{N}$, any n-bit string $x \in \{0,1\}^n$, $M(x) = f(x)$ and the computation $M(x)$ takes at most $T(n)$ steps of transitions.

Some theorems restrict the function $T(n) \geq n$ to be *time-constructable*, which means that there exists a TM M that for every $n \in \mathbb{N}$, $M(1^n)$ outputs the binary string of $T(n)$ in time at most $T(n)$, where 1^n denotes the n-bit string of all ones. Almost all functions in the big-O notation, such as $n, n \log n, n^4, 2^n, n^n$, are time-constructible.

We sometimes call a set $L \subseteq \{0,1\}^*$ of binary strings a *language*. In this course, we often use functions, problems, and languages interchangeably because a boolean function $f : \{0,1\}^* \to \{0,1\}$ defines a set $L = \{x \in \{0,1\}^* \mid f(x) = 1\}$, and vice versa.

3.1 Uncomputable Functions

With computability defined, we can easily show there exist *uncomputable functions*. It is a simple comparison of cardinalities: the set of all boolean functions $\{f : \{0,1\}^* \to \{0,1\}\}$ is uncountably infinite, but the set of all Turing Machines is countable. Because each TM computes either 1 boolean function (if the machine halts for all inputs) or 0 boolean functions (if the machine does not halt for some input), there must be infinitely many functions that are not computable by any TM.

3.2 Halting Problem

The function $HALT : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}$ is defined as

$$HALT(\alpha, x) = \begin{cases} 1 & \text{if } M_\alpha(x)\text{halts,} \\ 0 & \text{otherwise.} \end{cases}$$

This is the famous halting problem. Turing proved it uncomputable. Notice that we put no restriction on the TM—no matter how much time or space is given to a TM, it cannot solve the halting problem. See [AB09, Theorems 1.10 and 1.11] for proofs, where the diagonalization will be used again later in this course.

Theorem 2 (Turing)**.** *There is no TM M such that M computes $HALT$.*

The implication of Turing's halting problem is huge. It says that the fundamental property, whether a program halts, cannot be solved in general. It implies that almost all other properties about program behavior are uncomputable in general. It also implies Gödel's incompleteness, which was proved earlier than Turing's Theorem and will be covered briefly next week.

Acknowledgement

Replace this with people who helped with this note. If any publication is used, cite them like this [Tur37].

References

[AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[CBH20] Alex Churchill, Stella Biderman, and Austin Herrick. Magic: The Gathering is Turing complete. In *10th International Conference on Fun with Algorithms (FUN 2021)*, volume 157 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:19. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020.

[Chu36] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, April 1936.

[Kay07] Richard Kaye. Infinite versions of Minesweeper are Turing complete. Revised version of a manuscript originally published online in 2000. https://complexityexplorer.s3.amazonaws.com/Computation+in+CS/SFI+5.6c.pdf, May 2007.

[Tur37] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1937.